

**LAXMI CHARITABLE TRUST'S  
SHETH L.U.J COLLEGE OF ARTS & SIR M.V. COLLEGE OF SCIENCE AND  
COMMERCE  
DEPARTMENT OF INFORMATION TECHNOLOGY**

**BUSINESS ANALYTICS PROJECT**

**ONLINE RETAIL BUSINESS ANALYTICS:  
SALES, CUSTOMER AND DECISION ANALYSIS**

**Submitted By:**

**Name: Pranay Rajesh Bhadwalkar**

**Roll No.: T003**

**Class: TY B.Sc. IT**

**Division / Batch: B1**

**Subject: Business Analytics**

**Academic Year: 2026–2027**

## 1. Introduction

---

Business Analytics is the use of data, statistical methods and visualization to understand business performance and support better decisions. This project analyzes the Online Retail transaction dataset using Python and applies Exploratory Data Analysis, descriptive statistics, correlation, regression, probability, sampling, confidence intervals, bootstrapping, hypothesis testing, A/B testing, time-series analysis, simulation, ANOVA, factorial experiments, decision analysis, optimization, customer analysis and RFM measures.

## 2. Problem Statement and Objectives

---

An online retailer records a very large number of transactions across products, customers, countries and dates. Analysing this data manually makes it hard to find business patterns. This project uses Business Analytics to:

- Understand sales performance across countries, products and customers.
- Identify relationships between quantity, unit price and sales, and study sales trends over time.
- Test business assumptions statistically and evaluate decisions under uncertainty.
- Study customer purchasing behaviour using spending, purchase frequency and RFM measures.
- Provide useful business insights and recommendations.

## 3. Dataset and Tools

---

The main dataset is **OnlineRetail.csv** (541,909 records, 8 columns; 392,692 records remain after cleaning). Fields used include InvoiceNo, StockCode, Description, Quantity, InvoiceDate, UnitPrice, CustomerID and Country. One calculated column, **Total\_Sales** ( $\text{Quantity} \times \text{UnitPrice}$ ), is created. The dataset contains no product cost, so Total\_Sales represents sales revenue and is not treated as profit.

Additional datasets (A/B Testing, Simulation, Factorial Experiment, Decision Making and Optimization Input) are **synthetic and prepared for academic demonstration only**; they are not actual retailer experiments.

| Item               | Details                                                                                                                                                                                                        |
|--------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Language           | Python                                                                                                                                                                                                         |
| Environment        | Visual Studio Code, Jupyter Notebook                                                                                                                                                                           |
| Libraries          | Pandas, NumPy, Matplotlib, Seaborn, SciPy, Scikit-learn, Statsmodels (PuLP installed; optimization solved with SciPy)                                                                                          |
| Data format        | CSV                                                                                                                                                                                                            |
| Synthetic datasets | OnlineRetail_AB_Testing_Dataset.csv, OnlineRetail_Simulation_Dataset.csv, OnlineRetail_Factorial_Experiment_Dataset.csv, OnlineRetail_Decision_Making_Dataset.csv, OnlineRetail_Optimization_Input_Dataset.csv |

## 4. Setup, Data Loading and Preprocessing

---

### 4.1 Install and Import Libraries

Required libraries are installed and imported.

## Code

```
pip install pandas numpy matplotlib seaborn scipy scikit-learn statsmodels pulp openpyxl
jupyter
```

## Code

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from scipy import stats
from scipy.stats import ttest_1samp, f_oneway

from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

import statsmodels.api as sm
from statsmodels.formula.api import ols

import warnings
warnings.filterwarnings("ignore")

print("Libraries imported successfully!")
```

## Output

```
Libraries imported successfully!
```

## 4.2 Load Dataset

The dataset is loaded and the first rows are displayed.

## Code

```
df = pd.read_csv(
    "OnlineRetail.csv",
    encoding="latin1"
)

print("Dataset loaded successfully!")
print("Rows:", df.shape[0])
print("Columns:", df.shape[1])

display(df.head())
```

## Output

```
Dataset loaded successfully!
Rows: 541909
Columns: 8
```

|   | InvoiceNo | StockCode | Description                         | Quantity | \ |
|---|-----------|-----------|-------------------------------------|----------|---|
| 0 | 536365    | 85123A    | WHITE HANGING HEART T-LIGHT HOLDER  | 6        |   |
| 1 | 536365    | 71053     | WHITE METAL LANTERN                 | 6        |   |
| 2 | 536365    | 84406B    | CREAM CUPID HEARTS COAT HANGER      | 8        |   |
| 3 | 536365    | 84029G    | KNITTED UNION FLAG HOT WATER BOTTLE | 6        |   |
| 4 | 536365    | 84029E    | RED WOOLLY HOTTIE WHITE HEART.      | 6        |   |

|   | InvoiceDate      | UnitPrice | CustomerID | Country        |
|---|------------------|-----------|------------|----------------|
| 0 | 01-12-2010 08:26 | 2.55      | 17850.0    | United Kingdom |
| 1 | 01-12-2010 08:26 | 3.39      | 17850.0    | United Kingdom |
| 2 | 01-12-2010 08:26 | 2.75      | 17850.0    | United Kingdom |
| 3 | 01-12-2010 08:26 | 3.39      | 17850.0    | United Kingdom |
| 4 | 01-12-2010 08:26 | 3.39      | 17850.0    | United Kingdom |

### 4.3 Dataset Information

Column names, non-null counts and data types are checked.

#### Code

```
df.info()
```

#### Output

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 541909 entries, 0 to 541908
Data columns (total 8 columns):
 #   Column          Non-Null Count  Dtype  
---  -
 0   InvoiceNo       541909 non-null object  
 1   StockCode      541909 non-null object  
 2   Description     540455 non-null object  
 3   Quantity       541909 non-null int64   
 4   InvoiceDate     541909 non-null object  
 5   UnitPrice      541909 non-null float64  
 6   CustomerID     406829 non-null float64  
 7   Country        541909 non-null object  
dtypes: float64(2), int64(1), object(5)
memory usage: 33.1+ MB
```

### 4.4 Missing Values

Missing values are counted for every column.

#### Code

```
print("Missing Values:")
display(df.isnull().sum())
```

#### Output

```
Missing Values:
```

```
InvoiceNo      0
StockCode      0
Description    1454
Quantity       0
InvoiceDate    0
UnitPrice      0
CustomerID    135080
Country        0
dtype: int64
```

**Observation:** Description has 1,454 missing values and CustomerID has 135,080. Records without a CustomerID cannot be used for customer analysis, so they are removed during cleaning.

### 4.5 Duplicate Records

Duplicate rows are counted using `df.duplicated()`.

#### Code

```
print("Duplicate Records:", df.duplicated().sum())
```

#### Output

```
Duplicate Records: 5268
```

## 4.6 Data Types

Data types are checked so that numeric, categorical and date columns are used correctly. InvoiceDate is stored as text and is converted to datetime during cleaning.

### Code

```
display(df.dtypes)
```

### Output

```
InvoiceNo      object
StockCode      object
Description    object
Quantity       int64
InvoiceDate    object
UnitPrice      float64
CustomerID     float64
Country        object
dtype: object
```

## 4.7 Data Cleaning

Records without a CustomerID, records with non-positive Quantity or UnitPrice (returns and invalid entries) and duplicate rows are removed. InvoiceDate is converted to datetime; the dates are in day-month-year format (for example 01-12-2010), so dayfirst=True is used.

### Code

```
df = df.dropna(subset=["CustomerID"])

df = df[df["Quantity"] > 0]
df = df[df["UnitPrice"] > 0]

df = df.drop_duplicates()

df["InvoiceDate"] = pd.to_datetime(
    df["InvoiceDate"],
    dayfirst=True,
    errors="coerce"
)
```

**Observation:** After cleaning, 392,692 records remain out of the original 541,909.

## 4.8 Feature Engineering

The calculated column Total\_Sales is created for each transaction.

### Code

```
df["Total_Sales"] = (
    df["Quantity"] * df["UnitPrice"]
)
```

## 5. Descriptive Statistics

### 5.1 Summary of Key Numeric Variables

Count, mean, standard deviation, minimum, quartiles (the 50% value is the median) and maximum are calculated for Quantity, UnitPrice and Total\_Sales.

## Code

```
columns = [
    "Quantity",
    "UnitPrice",
    "Total Sales"
]

display(df[columns].describe())
```

## Output

```
count    392692.000000    392692.000000    392692.000000
mean      13.119702         3.125914         22.631500
std       180.492832        22.241836        311.099224
min         1.000000         0.001000         0.001000
25%        2.000000         1.250000         4.950000
50%        6.000000         1.950000        12.450000
75%       12.000000         3.750000        19.800000
max      80995.000000      8142.750000    168469.600000
```

**Observation:** The mean transaction value is 22.63 but the median is only 12.45, and the maximum is 168,469.60. Total\_Sales is therefore strongly right-skewed: most transactions are small and a few very large orders pull the average up. Quantity shows the same pattern (median 6, maximum 80,995).

## 5.2 Overall Sales Measures

### Code

```
print("Total Sales:", df["Total Sales"].sum())
print("Average Sales:", df["Total Sales"].mean())
print("Median Sales:", df["Total Sales"].median())
print("Maximum Sales:", df["Total Sales"].max())
print("Minimum Sales:", df["Total Sales"].min())
```

## Output

```
Total Sales: 8887208.894000001
Average Sales: 22.631499735161402
Median Sales: 12.45
Maximum Sales: 168469.6
Minimum Sales: 0.001
```

## 6. Exploratory Data Analysis

### 6.1 Total Sales by Country

A bar chart compares total sales across the ten highest-selling countries.

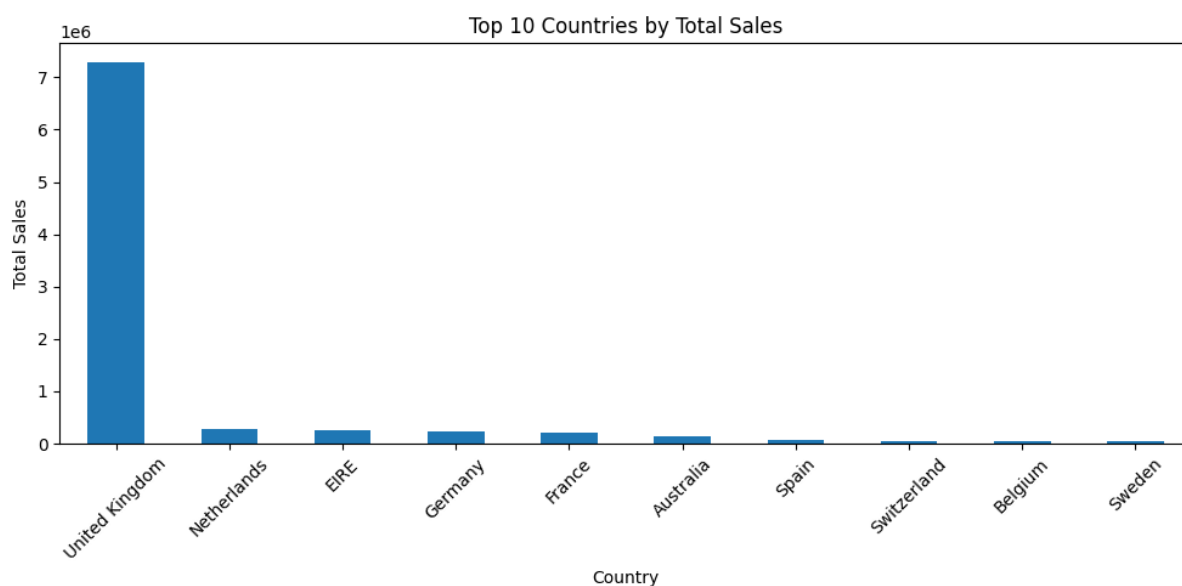
### Code

```
country_sales = (
    df.groupby("Country")["Total Sales"]
      .sum()
      .sort_values(ascending=False)
)

country_sales.head(10).plot(
    kind="bar",
    figsize=(10,5)
)

plt.title("Top 10 Countries by Total Sales")
plt.xlabel("Country")
plt.ylabel("Total Sales")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

## Output



**Observation:** The United Kingdom contributes about 82% of total sales (7.29 million out of 8.89 million), followed by Netherlands, EIRE, Germany and France.

## 6.2 Top Products by Sales

A bar chart shows the ten products with the highest total sales.

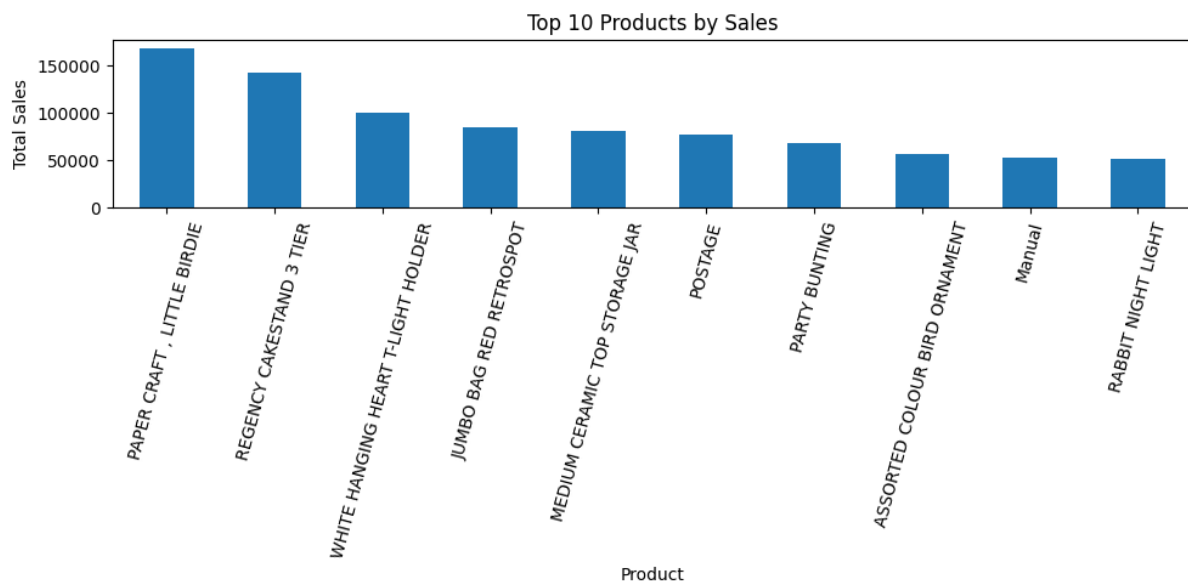
### Code

```
product_sales = (
    df.groupby("Description")["Total Sales"]
      .sum()
      .sort_values(ascending=False)
)

product_sales.head(10).plot(
    kind="bar",
    figsize=(10, 5)
)

plt.title("Top 10 Products by Sales")
plt.xlabel("Product")
plt.ylabel("Total Sales")
plt.xticks(rotation=75)
plt.tight_layout()
plt.show()
```

## Output



**Observation:** PAPER CRAFT, LITTLE BIRDIE has the highest sales (168,469.60), but this comes from a single very large order. REGENCY CAKESTAND 3 TIER (142,264.75) and WHITE HANGING HEART T-LIGHT HOLDER (100,392.10) follow.

### 6.3 Total Sales Histogram

The histogram shows the distribution and shape of Total\_Sales.

#### Code

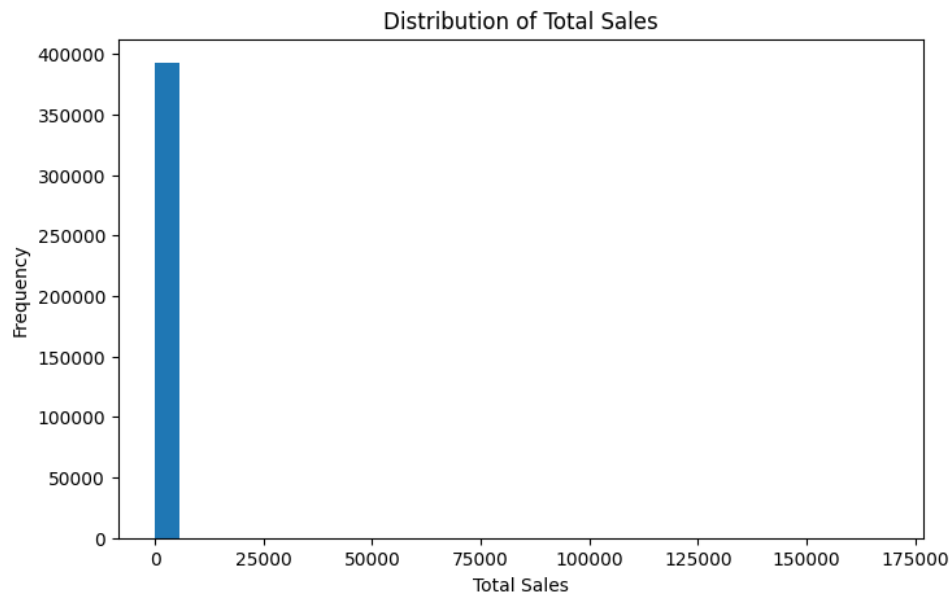
```
plt.figure(figsize=(8,5))

plt.hist(
    df["Total_Sales"],
    bins=30
)

plt.title("Distribution of Total Sales")
plt.xlabel("Total Sales")
plt.ylabel("Frequency")

plt.show()
```

## Output



**Observation:** Almost all transactions fall in the first bar and the distribution has a very long right tail.

## 6.4 Quantity Histogram

The histogram shows the distribution of Quantity.

### Code

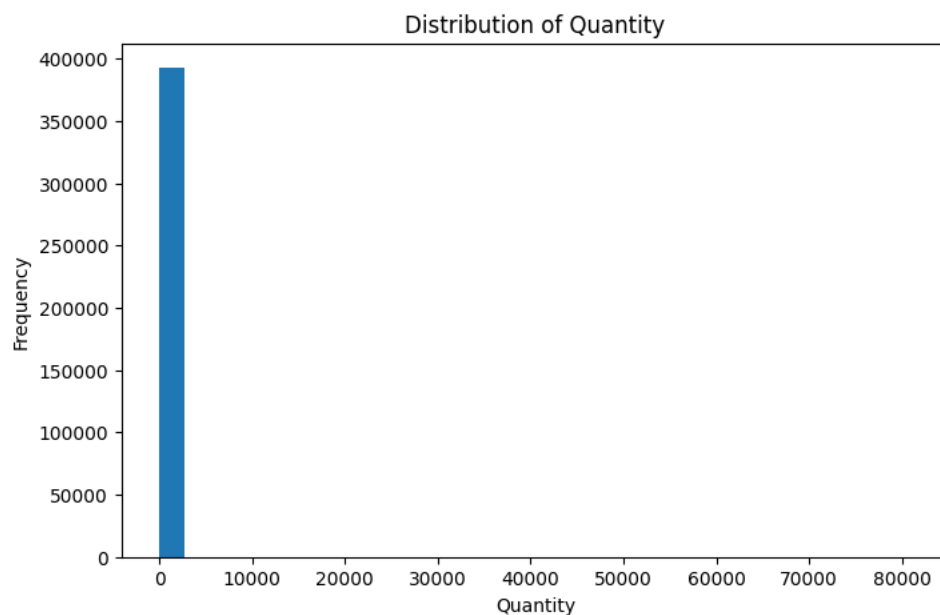
```
plt.figure(figsize=(8,5))

plt.hist(
    df["Quantity"],
    bins=30
)

plt.title("Distribution of Quantity")
plt.xlabel("Quantity")
plt.ylabel("Frequency")

plt.show()
```

## Output



## 6.5 Total Sales Box Plot

The box plot shows the median, spread and outliers of Total\_Sales.

### Code

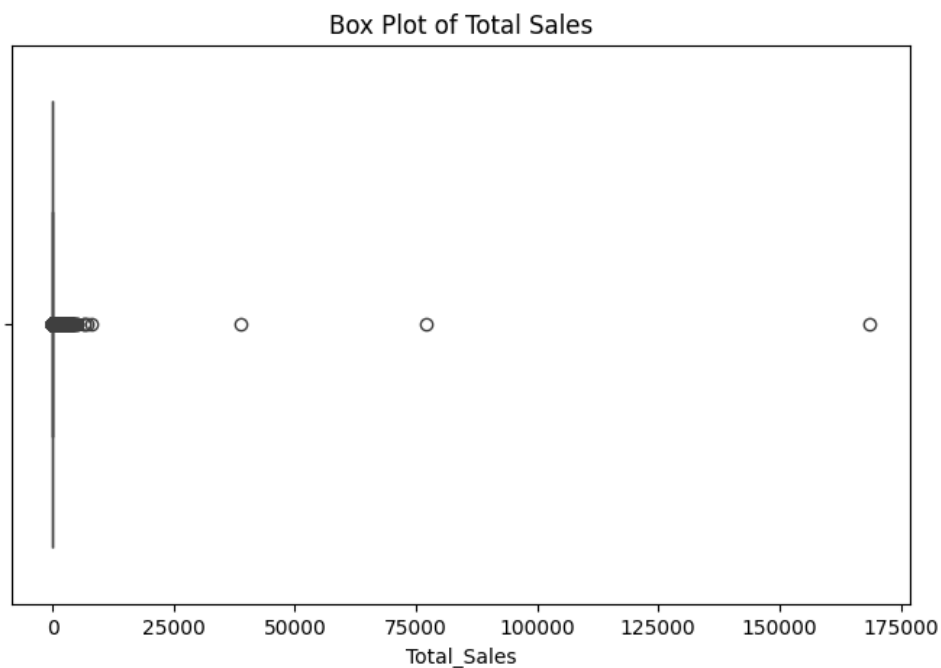
```
plt.figure(figsize=(8,5))

sns.boxplot(
    x=df["Total Sales"]
)

plt.title("Box Plot of Total Sales")

plt.show()
```

### Output



**Observation:** The box is compressed near zero and many outliers extend to the right, confirming the skewed distribution.

## 6.6 Transactions by Country

A bar chart shows the number of transactions for the ten most frequent countries.

### Code

```
country_count = df["Country"].value_counts().head(10)

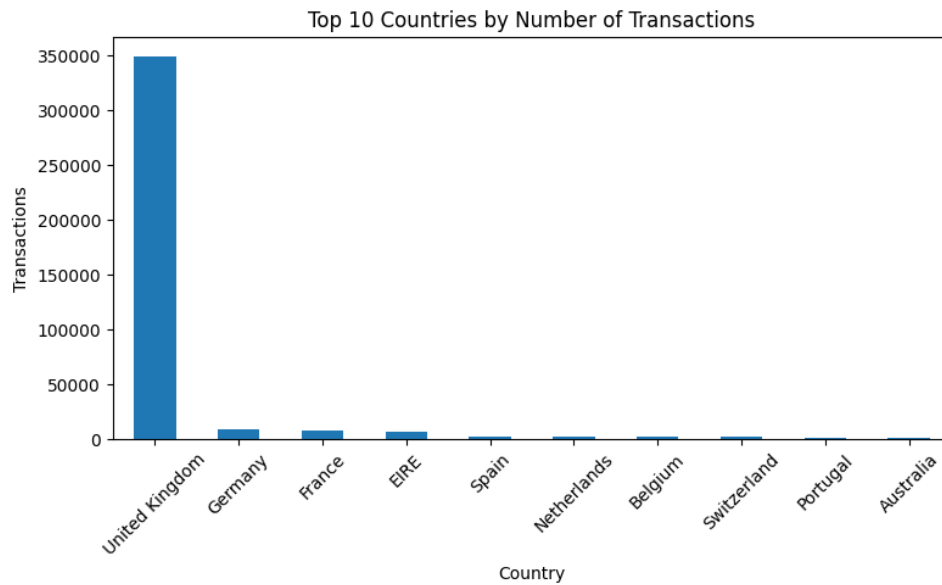
plt.figure(figsize=(8,5))

country_count.plot(kind="bar")

plt.title("Top 10 Countries by Number of Transactions")
plt.xlabel("Country")
plt.ylabel("Transactions")

plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

## Output



**Observation:** The United Kingdom accounts for 349,203 of the 392,692 transactions (88.9%), far ahead of Germany (9,025), France (8,326) and EIRE (7,226).

## 6.7 Quantity vs Total Sales

The scatter plot shows whether larger quantities lead to higher sales.

### Code

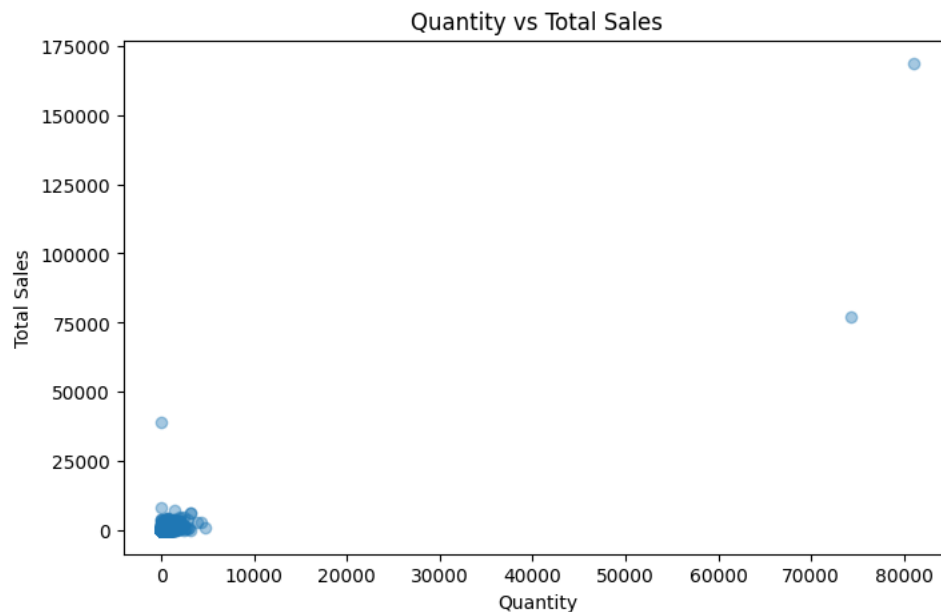
```
plt.figure(figsize=(8,5))

plt.scatter(
    df["Quantity"],
    df["Total Sales"],
    alpha=0.4
)

plt.title("Quantity vs Total Sales")
plt.xlabel("Quantity")
plt.ylabel("Total Sales")

plt.show()
```

## Output



**Observation:** Total\_Sales rises with Quantity, with a few extreme orders standing far apart from the rest.

## 7. Correlation Analysis

### 7.1 Correlation Matrix

The correlation matrix measures linear relationships between Quantity, UnitPrice and Total\_Sales.

#### Code

```
numeric_columns = [
    "Quantity",
    "UnitPrice",
    "Total Sales"
]

correlation = df[numeric_columns].corr()

display(correlation)
```

## Output

|             | Quantity  | UnitPrice | Total Sales |
|-------------|-----------|-----------|-------------|
| Quantity    | 1.000000  | -0.004578 | 0.914451    |
| UnitPrice   | -0.004578 | 1.000000  | 0.081619    |
| Total Sales | 0.914451  | 0.081619  | 1.000000    |

### 7.2 Correlation Heatmap

#### Code

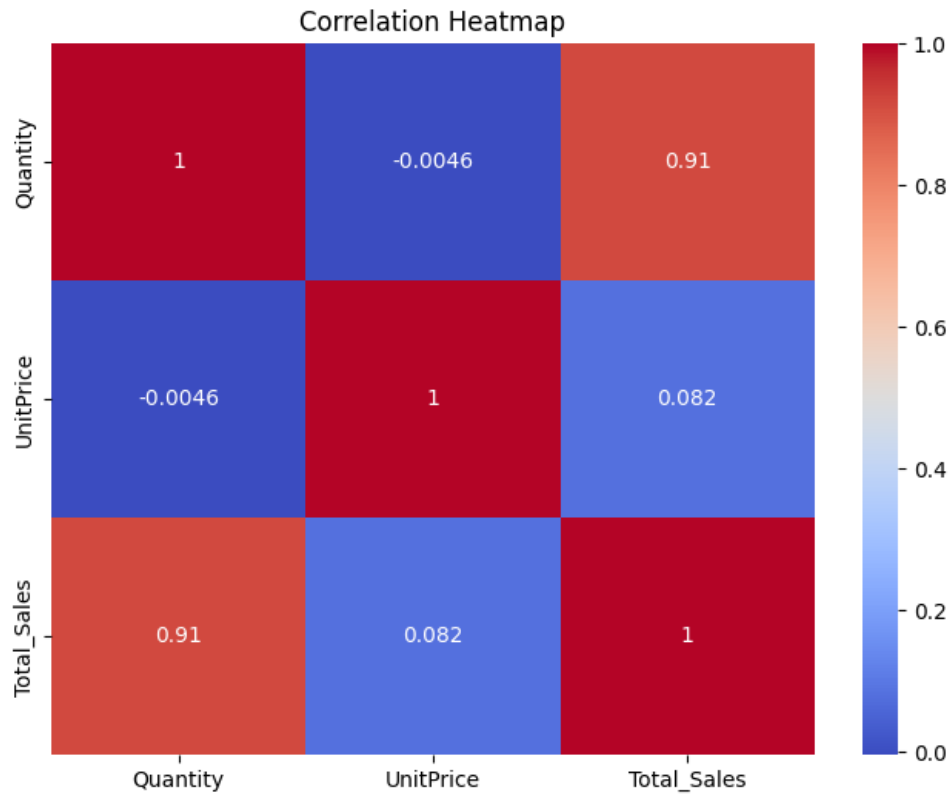
```
plt.figure(figsize=(8,6))

sns.heatmap(
    correlation,
    annot=True,
    cmap="coolwarm"
)

plt.title("Correlation Heatmap")

plt.show()
```

## Output



**Observation:** Quantity and Total\_Sales have a strong positive correlation (0.91). UnitPrice has only a weak correlation with Total\_Sales (0.08) and almost none with Quantity ( $-0.005$ ).

## 8. Regression Analysis

Business question: does Quantity sold affect Total\_Sales? A simple linear regression is fitted with Quantity as the independent variable and Total\_Sales as the dependent variable.

## Code

```
regression_df = df[
    ["Quantity", "Total Sales"]
].dropna()

X = regression_df[["Quantity"]]
y = regression_df["Total Sales"]

model = LinearRegression()

model.fit(X, y)

prediction = model.predict(X)

print("Coefficient:", model.coef_[0])
print("Intercept:", model.intercept_)

print(
    "R-squared:",
    r2_score(y, prediction)
)

print(
    "MSE:",
    mean_squared_error(y, prediction)
)

print(
    "RMSE:",
    np.sqrt(mean_squared_error(y, prediction))
)
```

## Output

```
Coefficient: 1.5761563835138797
Intercept: 1.9527977494416113
R-squared: 0.8362202347320562
MSE: 15851.012007467581
RMSE: 125.90080225108807
```

**Observation:** Each additional unit is associated with about 1.58 more sales, and the model explains 83.6% of the variation in Total\_Sales ( $R^2 = 0.836$ ). The RMSE of 125.90 is large compared with the average sale because of extreme orders. Since Total\_Sales is calculated as Quantity  $\times$  UnitPrice, part of this relationship is built into the data.

## 9. Probability

Probability is calculated as favourable records divided by total records. The question is: what is the probability that a randomly selected transaction comes from the United Kingdom?

### Code

```
total_records = len(df)

uk_records = (
    df["Country"] == "United Kingdom"
).sum()

probability_uk = (
    uk_records / total_records
)

print(
    "Probability of UK transaction:",
    probability_uk
)
```

## Output

```
Probability of UK transaction: 0.8892541737544947
```

**Observation:** The probability is about 0.889, so roughly 89 out of every 100 transactions come from the United Kingdom.

## 10. Sampling and Central Limit Theorem

---

Repeated random samples of sizes 30, 50 and 100 (100 repetitions each) are drawn from Total\_Sales, and the mean and standard error of the sample means are compared.

### Code

```
sales = df["Total Sales"].dropna()

sample_sizes = [30, 50, 100]

for size in sample_sizes:
    sample_means = []

    for i in range(100):
        sample = sales.sample(
            size,
            replace=True
        )

        sample_means.append(
            sample.mean()
        )

    print("Sample Size:", size)
    print(
        "Mean of Sample Means:",
        np.mean(sample_means)
    )

    print(
        "Standard Error:",
        np.std(
            sample_means,
            ddof=1
        )
    )
```

## Output

```
Sample Size: 30
Mean of Sample Means: 22.799536666666665
Standard Error: 28.629916518418568
Sample Size: 50
Mean of Sample Means: 19.540036
Standard Error: 5.615131259995063
Sample Size: 100
Mean of Sample Means: 29.626795
Standard Error: 76.91705051181071
```

**Observation:** The averages of the sample means stay close to the population mean of 22.63. The standard errors (28.63, 5.62 and 76.92) do not fall steadily with sample size because Total\_Sales has extreme outliers that appear randomly in some samples; with more repetitions the standard error would be expected to shrink as the sample size grows.

## 11. 95% Confidence Interval

---

A 95% confidence interval for the mean Total\_Sales is calculated using the t-distribution.

## Code

```
sales = df["Total Sales"].dropna()

mean = sales.mean()

n = len(sales)

standard error = stats.sem(sales)

lower, upper = stats.t.interval(
    0.95,
    df=n-1,
    loc=mean,
    scale=standard error
)

print("Mean Sales:", mean)

print(
    "95% Confidence Interval:",
    (lower, upper)
)
```

## Output

```
Mean Sales: 22.631499735161402
95% Confidence Interval: (np.float64(21.658478471778487), np.float64(23.604520998544317))
```

**Observation:** We are 95% confident that the true mean transaction value lies between about 21.66 and 23.60.

## 12. Bootstrapping

Bootstrapping resamples the data with replacement 1,000 times to estimate the sampling distribution of the mean.

## Code

```
sales = df["Total Sales"].dropna().to_numpy()

bootstrap_means = []

for _ in range(1000):
    sample = np.random.choice(
        sales,
        size=len(sales),
        replace=True
    )

    bootstrap_means.append(
        sample.mean()
    )

lower, upper = np.percentile(
    bootstrap_means,
    [2.5, 97.5]
)

print(
    "Bootstrap Mean:",
    np.mean(bootstrap_means)
)

print(
    "Bootstrap 95% CI:",
    (lower, upper)
)
```

## Output

```
Bootstrap Mean: 22.647754859678834
Bootstrap 95% CI: (np.float64(21.855508334076074), np.float64(23.695747015409022))
```

**Observation:** The bootstrap mean (22.65) and interval (21.86 to 23.70) are very close to the t-based confidence interval, which supports the result.

## 13. Hypothesis Testing

A one-sample t-test checks whether the mean Total\_Sales differs from an assumed benchmark of 100.  $H_0$ : mean Total\_Sales = 100;  $H_1$ : mean Total\_Sales is different.  $\alpha = 0.05$ .

### Code

```
test value = 100

sales = df["Total_Sales"].dropna()

result = ttest_1samp(
    sales,
    popmean=test value
)

print("H0: Mean Total Sales =", test value)
print("H1: Mean Total Sales is different")

print("t-statistic:", result.statistic)
print("p-value:", result.pvalue)

if result.pvalue < 0.05:
    print("Decision: Reject H0")
else:
    print("Decision: Fail to reject H0")
```

## Output

```
H0: Mean Total Sales = 100
H1: Mean Total Sales is different
t-statistic: -155.84442720109215
p-value: 0.0
Decision: Reject H0
```

**Observation:** The p-value is practically zero, so  $H_0$  is rejected. The mean transaction value (22.63) is significantly different from the assumed benchmark of 100; the benchmark is an assumed business target.

## 14. A/B Testing

Variant A (“Shop Now”) and Variant B (“Get 20% Off”) are compared with a two-proportion z-test.  $H_0$ : no difference in conversion rates;  $H_1$ : there is a difference. The data is a synthetic academic experiment.

### 14.1 Conversion Summary

#### Code

```
ab = pd.read_csv("OnlineRetail_AB_Testing_Dataset.csv")

display(ab.head())

display(ab.groupby("Variant")["Converted"].agg(["sum", "count", "mean"]))
```

## Output

|   | Visitor ID | Variant | Message     | Converted |
|---|------------|---------|-------------|-----------|
| 0 | V0001      | B       | Get 20% Off | 0         |
| 1 | V0002      | A       | Shop Now    | 0         |
| 2 | V0003      | B       | Get 20% Off | 0         |
| 3 | V0004      | A       | Shop Now    | 0         |
| 4 | V0005      | B       | Get 20% Off | 0         |

|         | sum | count | mean  |
|---------|-----|-------|-------|
| Variant |     |       |       |
| A       | 110 | 1000  | 0.110 |
| B       | 145 | 1000  | 0.145 |

## 14.2 Two-Proportion z-Test

### Code

```
from statsmodels.stats.proportion import proportions_ztest

summary = ab.groupby("Variant")["Converted"].agg(["sum", "count"])

z_stat, p_value = proportions_ztest(summary["sum"].to_numpy(), summary["count"].to_numpy())

print("Z-statistic:", z_stat)

print("P-value:", p_value)

print("Decision:", "Significant difference" if p_value < 0.05 else "No significant difference detected")
```

## Output

```
Z-statistic: -2.346471244078323
P-value: 0.018952128575399498
Decision: Significant difference
```

**Observation:** Variant B converts 14.5% of visitors against 11.0% for Variant A. The p-value (0.019) is below 0.05, so the difference is statistically significant and the “Get 20% Off” message performs better in this experiment.

## 15. Time-Series Analysis

---

### 15.1 Monthly Sales

The InvoiceDate is converted to datetime and sales are summed by month.

## Code

```
df["InvoiceDate"] = pd.to_datetime(
    df["InvoiceDate"],
    errors="coerce"
)

dated_df = df.dropna(
    subset=["InvoiceDate"]
).copy()

monthly_sales = (
    dated_df
    .groupby(
        dated_df["InvoiceDate"].dt.to_period("M")
    )["Total_Sales"]
    .sum()
)

monthly_sales.index = (
    monthly_sales.index.to_timestamp()
)
```

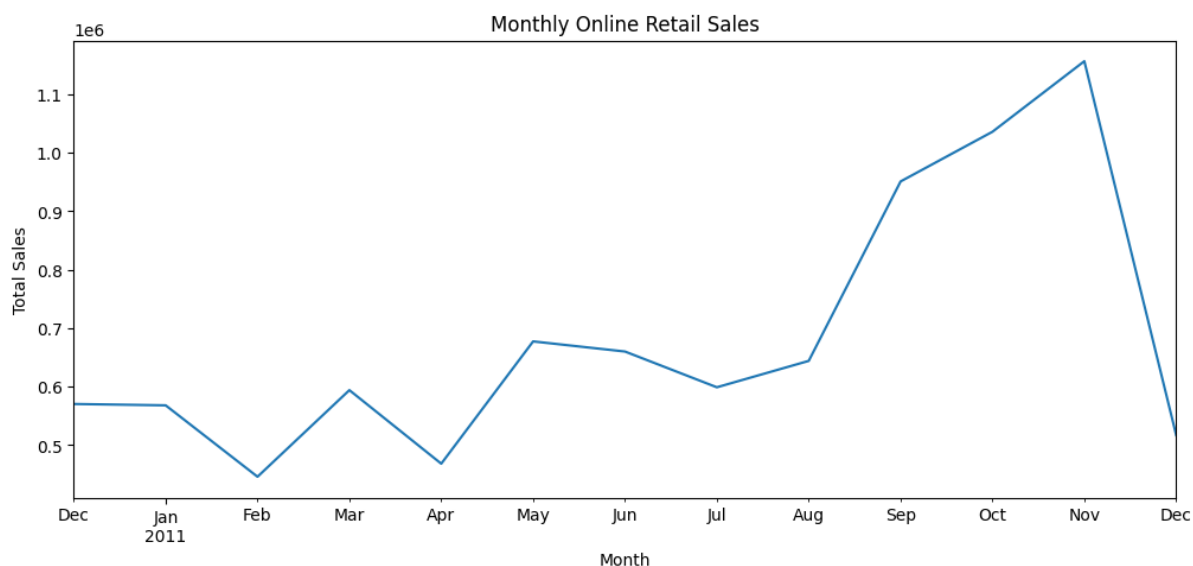
## Code

```
monthly_sales.plot(
    figsize=(12, 5)
)

plt.title("Monthly Online Retail Sales")
plt.xlabel("Month")
plt.ylabel("Total Sales")

plt.show()
```

## Output



**Observation:** Monthly sales run from December 2010 to December 2011. Sales are fairly steady until mid-2011, then rise sharply from September, peaking in November 2011 at about 1.16 million. December 2011 is lower because the data ends on 9 December, so it is only a partial month.

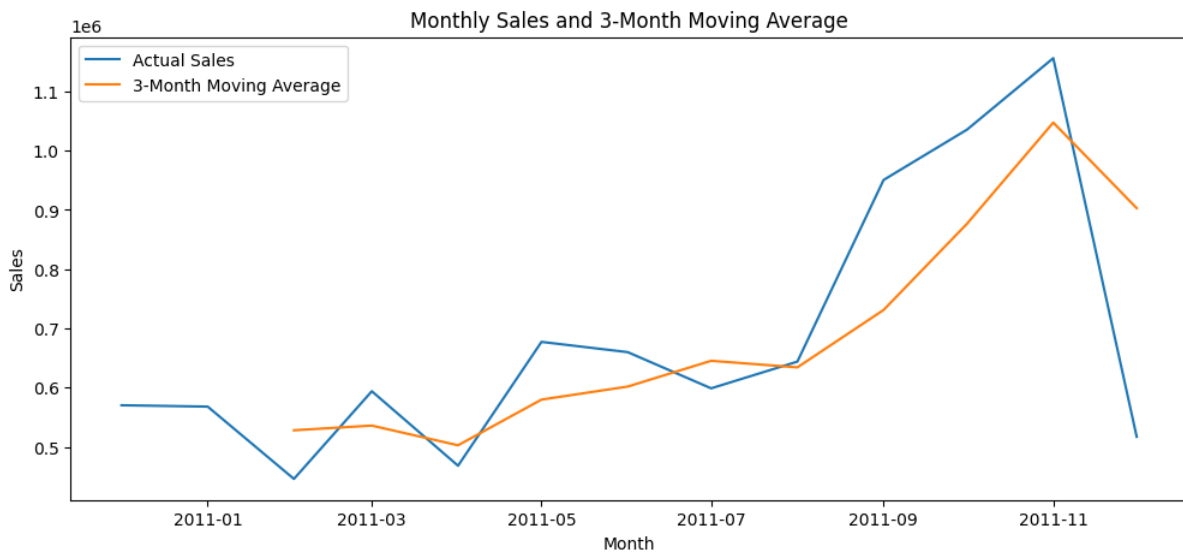
## 15.2 Three-Month Moving Average

A 3-month moving average smooths short-term changes to show the general trend.

## Code

```
moving_average = (  
    monthly_sales  
    .rolling(window=3)  
    .mean()  
)  
  
plt.figure(figsize=(12,5))  
  
plt.plot(  
    monthly_sales.index,  
    monthly_sales.values,  
    label="Actual Sales"  
)  
  
plt.plot(  
    moving_average.index,  
    moving_average.values,  
    label="3-Month Moving Average"  
)  
  
plt.title(  
    "Monthly Sales and 3-Month Moving Average"  
)  
  
plt.xlabel("Month")  
plt.ylabel("Sales")  
  
plt.legend()  
  
plt.show()
```

## Output



**Observation:** The moving average removes the month-to-month swings and shows a clear upward trend in the second half of 2011, which is useful for short-term inventory planning.

## 16. Simulation

Low, Normal and High demand scenarios are simulated and their average outcomes compared. The Online Retail data has no scenario data, so a synthetic dataset is used, with baselines taken from the cleaned data (average daily demand of about 16,892 units and average unit price of about 1.75).

## 16.1 Load Simulation Dataset

### Code

```
simulation = pd.read_csv("OnlineRetail Simulation Dataset.csv")

print("Available columns:", simulation.columns.tolist())

display(simulation.head())
```

### Output

```
Available columns: ['Simulation ID', 'Scenario', 'Demand', 'Unit Price', 'Sales']
```

|   | Simulation ID | Scenario | Demand | Unit Price | Sales    |
|---|---------------|----------|--------|------------|----------|
| 0 | 1             | Low      | 14131  | 1.80       | 25444.79 |
| 1 | 2             | Low      | 11406  | 1.96       | 22368.56 |
| 2 | 3             | Low      | 15035  | 1.76       | 26515.54 |
| 3 | 4             | Low      | 15420  | 1.85       | 28474.43 |
| 4 | 5             | Low      | 9559   | 1.44       | 13788.61 |

## 16.2 Average Outcome by Scenario

### Code

```
if "Scenario" in simulation.columns:

    numeric_cols = simulation.select_dtypes(include="number").columns.tolist()

    numeric_cols = [c for c in numeric_cols if c != "Simulation_ID"]

    display(simulation.groupby("Scenario")[numeric_cols].mean().loc[["Low", "Normal",
"High"]])

else:

    print("No Scenario column found. Check the available column names above.")
```

### Output

|          | Demand   | Unit Price | Sales      |
|----------|----------|------------|------------|
| Scenario |          |            |            |
| Low      | 13451.89 | 1.75325    | 23562.2093 |
| Normal   | 16719.38 | 1.74470    | 29201.9300 |
| High     | 20181.58 | 1.75180    | 35352.0699 |

**Observation:** Average demand rises from 13,452 (Low) to 16,719 (Normal) and 20,182 (High), and average sales rise from 23,562 to 29,202 and 35,352. The unit price stays at about 1.75 in every scenario, so sales change mainly because of demand.

## 17. ANOVA

One-way ANOVA tests whether mean Total\_Sales differs across the five countries with the most transactions (United Kingdom, Germany, France, EIRE and Spain).  $H_0$ : all group means are equal;  $H_1$ : at least one group mean differs.  $\alpha = 0.05$ .

## 17.1 One-Way ANOVA

### Code

```
top_countries = (
    df["Country"]
    .value_counts()
    .head(5)
    .index
)

anova_df = df[
    df["Country"].isin(top_countries)
].copy()

groups = []

for country in top_countries:

    values = anova_df[
        anova_df["Country"] == country
    ][["Total Sales"].dropna()

    groups.append(values)

f_stat, p_value = f_oneway(*groups)

print("F-statistic:", f_stat)
print("P-value:", p_value)

if p_value < 0.05:
    print("Significant difference detected")
else:
    print("No significant difference detected")
```

### Output

```
F-statistic: 5.171925593881521
P-value: 0.00036524539626795534
Significant difference detected
```

**Observation:** The p-value (0.00037) is below 0.05, so  $H_0$  is rejected: average transaction sales differ significantly between at least two of these countries.

## 18. Factorial Experiment ( $2 \times 3$ )

Factors: Sales Channel (Online, Store) and Promotion Level (Low, Medium, High). Response: Sales. Each observation is an experimental unit, with 20 replications per combination. The Online Retail data is observational, so a synthetic experiment dataset is used.

### 18.1 Load Factorial Dataset

#### Code

```
factorial = pd.read_csv("OnlineRetail Factorial Experiment Dataset.csv")

print("Available columns:", factorial.columns.tolist())

display(factorial.head())
```

#### Output

```
Available columns: ['Experiment ID', 'Sales Channel', 'Promotion Level', 'Sales']
```

|   | Experiment ID | Sales Channel | Promotion Level | Sales    |
|---|---------------|---------------|-----------------|----------|
| 0 | 1             | Online        | Low             | 29914.15 |
| 1 | 2             | Online        | Low             | 25880.05 |
| 2 | 3             | Online        | Low             | 31251.35 |
| 3 | 4             | Online        | Low             | 31821.69 |
| 4 | 5             | Online        | Low             | 23146.89 |

## 18.2 Mean Sales Table and Interaction Plot

### Code

```
if {"Sales Channel", "Promotion Level", "Sales"}.issubset(factorial.columns):

    order = ["Low", "Medium", "High"]

    display(factorial.groupby(["Sales_Channel",
                              "Promotion Level"])["Sales"].mean().unstack()[order])

    sns.pointplot(data=factorial, x="Promotion Level", y="Sales", hue="Sales Channel",
                  order=order)

    plt.title("Factorial Experiment: Sales by Promotion Level and Channel")

    plt.tight_layout()

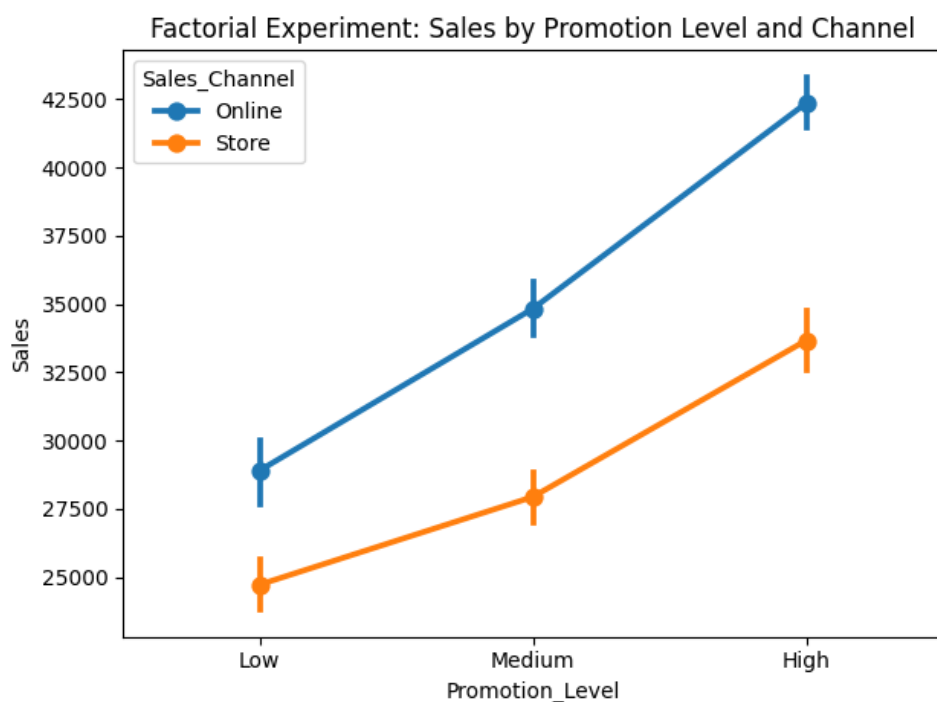
    plt.show()

else:

    print("Expected columns not found; check the column list in the previous cell.")
```

### Output

| Promotion Level | Low        | Medium    | High       |
|-----------------|------------|-----------|------------|
| Sales_Channel   |            |           |            |
| Online          | 28901.2025 | 34828.386 | 42357.9795 |
| Store           | 24721.0845 | 27937.302 | 33666.8105 |



**Observation:** Online sales are higher than Store sales at every promotion level, and sales increase as the promotion level goes from Low to High in both channels.

## 18.3 Two-Way ANOVA

Two-way ANOVA tests the effect of each factor and of their interaction.

### Code

```
import statsmodels.api as sm
from statsmodels.formula.api import ols

factorial model = ols("Sales ~ C(Sales Channel) * C(Promotion Level)", data=factorial).fit()

display(sm.stats.anova_lm(factorial model, typ=2))
```

### Output

```
              sum sq      df      F      \
C(Sales Channel)      1.301838e+09      1.0    234.657545
C(Promotion Level)      2.537592e+09      2.0    228.701780
C(Sales Channel):C(Promotion Level)      1.031307e+08      2.0      9.294710
Residual              6.324514e+08    114.0         NaN

              PR(>F)
C(Sales Channel)      1.921085e-29
C(Promotion Level)      1.252647e-40
C(Sales Channel):C(Promotion Level)      1.822094e-04
Residual              NaN
```

**Observation:** All three p-values are below 0.05: Sales Channel, Promotion Level and their interaction all have a significant effect on Sales. The interaction means the benefit of a higher promotion level is not identical in the two channels.

## 19. Decision Making / Expected Value

Expected Value = Probability × Payoff. The alternative with the highest expected value is preferred. The payoffs are assumed values in INR for academic demonstration.

### 19.1 Load Decision Dataset

#### Code

```
decision = pd.read_csv("OnlineRetail Decision Making Dataset.csv")

print("Available columns:", decision.columns.tolist())

display(decision)
```

### Output

```
Available columns: ['Alternative', 'Outcome', 'Probability', 'Payoff INR']
```

|    | Alternative          | Outcome           | Probability | Payoff INR |
|----|----------------------|-------------------|-------------|------------|
| 0  | Increase Advertising | Strong response   | 0.30        | 900000     |
| 1  | Increase Advertising | Moderate response | 0.50        | 450000     |
| 2  | Increase Advertising | Weak response     | 0.20        | -150000    |
| 3  | Offer Discount       | Strong response   | 0.25        | 800000     |
| 4  | Offer Discount       | Moderate response | 0.45        | 400000     |
| 5  | Offer Discount       | Weak response     | 0.30        | -200000    |
| 6  | Launch New Product   | Strong response   | 0.20        | 1500000    |
| 7  | Launch New Product   | Moderate response | 0.40        | 500000     |
| 8  | Launch New Product   | Weak response     | 0.40        | -500000    |
| 9  | Expand Market        | Strong response   | 0.25        | 1200000    |
| 10 | Expand Market        | Moderate response | 0.45        | 600000     |
| 11 | Expand Market        | Weak response     | 0.30        | -400000    |

## 19.2 Expected Value by Alternative

### Code

```
if {"Alternative", "Probability", "Payoff INR"}.issubset(decision.columns):

    decision["Expected Value"] = decision["Probability"] * decision["Payoff INR"]

    values =
decision.groupby("Alternative")["Expected Value"].sum().sort_values(ascending=False)

    display(values)

    print("Best alternative:", values.idxmax())

    print("Expected value:", values.max())

else:

    print("Expected columns not found; check the column list above.")
```

### Output

```
Alternative
Increase Advertising    465000.0
Expand Market          450000.0
Offer Discount          320000.0
Launch New Product      300000.0
Name: Expected Value, dtype: float64
```

```
Best alternative: Increase Advertising
Expected value: 465000.0
```

**Observation:** Increase Advertising has the highest expected value (465,000), followed by Expand Market (450,000), Offer Discount (320,000) and Launch New Product (300,000), so Increase Advertising is the preferred alternative.

## 20. Optimization

The optimization inputs hold product-level profit and machine and labour hours. The objective is to maximize total profit, subject to the available machine hours (120), the available labour hours (100) and product quantities  $\geq 0$  (whole units). The resource limits are assumed values. The model is solved as an integer linear program with SciPy's milp solver.

### 20.1 Optimization Inputs

#### Code

```
optimization = pd.read_csv("OnlineRetail_Optimization_Input_Dataset.csv")

display(optimization)

if "Product" in optimization.columns:

    print("Products:", optimization["Product"].tolist())
```

#### Output

|   | Product   | Profit | Machine Hours | Labour Hours |
|---|-----------|--------|---------------|--------------|
| 0 | Product A | 40     | 2             | 1            |
| 1 | Product B | 30     | 1             | 2            |
| 2 | Product C | 50     | 3             | 2            |

```
Products: ['Product A', 'Product B', 'Product C']
```

## 20.2 Optimization Model

### Code

```
import numpy as np
from scipy.optimize import milp, LinearConstraint, Bounds

machine_available = 120    # assumed machine hours available
labour_available = 100     # assumed labour hours available

products = optimization["Product"].tolist()

# milp minimizes, so profit is negated to maximize it
c = -optimization["Profit"].to_numpy(dtype=float)

A = np.array([optimization["Machine_Hours"], optimization["Labour_Hours"]], dtype=float)

result = milp(
    c=c,
    constraints=LinearConstraint(A, ub=[machine_available, labour_available]),
    integrality=np.ones(len(products)), # whole units only
    bounds=Bounds(lb=0),                # quantity >= 0
)

units = np.round(result.x).astype(int)

print("Status:", "Optimal" if result.success else "Not solved")

for p, u in zip(products, units):
    print(p, "->", int(u), "units")

print("Maximum Total Profit:", -result.fun)
print("Machine hours used:", (A[0] * units).sum(), "of", machine_available)
print("Labour hours used:", (A[1] * units).sum(), "of", labour_available)
```

### Output

```
Status: Optimal
Product A -> 47 units
Product B -> 26 units
Product C -> 0 units
Maximum Total Profit: 2660.0
Machine hours used: 120.0 of 120
Labour hours used: 99.0 of 100
```

**Observation:** The best plan is to produce 47 units of Product A and 26 units of Product B and none of Product C, giving a maximum profit of 2,660. All 120 machine hours and 99 of the 100 labour hours are used.

## 21. Customer Analysis

### 21.1 Customer Total Spending

Total\_Sales is summed for each customer and the ten highest-spending customers are listed.

### Code

```
customer_sales = (
    df.groupby("CustomerID")
      ["Total Sales"]
      .sum()
      .sort_values(ascending=False)
)

display(
    customer_sales.head(10)
)
```

## Output

```
CustomerID
14646.0    280206.02
18102.0    259657.30
17450.0    194390.79
16446.0    168472.50
14911.0    143711.17
12415.0    124914.53
14156.0    117210.08
17511.0     91062.38
16029.0     80850.84
12346.0     77183.60
Name: Total Sales, dtype: float64
```

**Observation:** Customer 14646 is the highest spender (280,206.02), followed by 18102 (259,657.30) and 17450 (194,390.79).

## 21.2 Purchase Frequency

The number of unique invoices is counted for each customer.

### Code

```
customer frequency = (
    df.groupby("CustomerID")
      ["InvoiceNo"]
      .nunique()
      .sort values(ascending=False)
)

display(
    customer frequency.head(10)
)
```

## Output

```
CustomerID
12748.0    209
14911.0    201
17841.0    124
13089.0     97
14606.0     93
15311.0     91
12971.0     86
14646.0     73
16029.0     63
13408.0     62
Name: InvoiceNo, dtype: int64
```

**Observation:** Customer 12748 purchased most often (209 invoices), followed by 14911 (201) and 17841 (124). Customers 14646, 14911 and 16029 appear in both top-ten lists, so they are both frequent and high-value.

## 21.3 RFM Measures

RFM stands for Recency (days since the last purchase), Frequency (number of invoices) and Monetary (total spending). The latest invoice date in the data is used as the reference date.

## Code

```
reference_date = df["InvoiceDate"].max()

rfm = df.groupby("CustomerID").agg({

    "InvoiceDate": lambda x:
        (reference_date - x.max()).days,

    "InvoiceNo": "nunique",

    "Total Sales": "sum"

})

rfm.columns = [
    "Recency",
    "Frequency",
    "Monetary"
]

display(rfm.head())
```

## Output

| CustomerID | Recency | Frequency | Monetary |
|------------|---------|-----------|----------|
| 12346.0    | 325     | 1         | 77183.60 |
| 12347.0    | 1       | 7         | 4310.00  |
| 12348.0    | 74      | 4         | 1797.24  |
| 12349.0    | 18      | 1         | 1757.55  |
| 12350.0    | 309     | 1         | 334.40   |

**Observation:** For example, customer 12347 bought 1 day before the reference date, 7 times, spending 4,310, whereas customer 12346 bought only once, 325 days earlier, with a single large purchase of 77,183.60.

## 22. Final Business Summary

---

### 22.1 KPI Summary

#### Code

```
print("ONLINE RETAIL BUSINESS ANALYTICS")
print("-----")

print(
    "Total Sales:",
    round(df["Total_Sales"].sum(), 2)
)

print(
    "Total Units Sold:",
    round(df["Quantity"].sum(), 2)
)

print(
    "Average Transaction Sales:",
    round(df["Total_Sales"].mean(), 2)
)

print(
    "Unique Customers:",
    df["CustomerID"].nunique()
)

print(
    "Unique Products:",
    df["StockCode"].nunique()
)

print(
    "Countries:",
    df["Country"].nunique()
)
```

#### Output

```
ONLINE RETAIL BUSINESS ANALYTICS
-----
Total Sales: 8887208.89
Total Units Sold: 5152002
Average Transaction Sales: 22.63
Unique Customers: 4338
Unique Products: 3665
Countries: 37
```

## 23. Business Recommendations

---

- Focus on the best-selling products and keep them well stocked, especially before the September–November peak.
- Identify and retain high-value, frequent customers with loyalty offers.
- The United Kingdom provides about 82% of sales; develop strategies to grow sales in other countries such as the Netherlands, EIRE, Germany and France.
- Use monthly sales trends and the 3-month moving average for inventory and demand planning.
- Use statistical testing (such as the A/B test) before making major promotional decisions.
- Use simulation and optimization to prepare for different demand scenarios and to allocate limited resources.

These recommendations are based on the results above; the A/B, simulation, factorial, decision and optimization results use synthetic academic data.

## 24. Conclusion

---

This project applied Business Analytics techniques to an Online Retail transaction dataset, from preprocessing and exploratory analysis to statistical inference, time-series analysis, simulation, ANOVA, factorial experiments, decision analysis, optimization and customer analysis. Sales are heavily concentrated in the United Kingdom and in a small number of very large orders, quantity strongly drives sales, and sales grow strongly towards the end of the year. The results convert raw transaction data into insights that can support decisions on products, customers, countries and resource planning.

## 25. References

---

- Python, Pandas, NumPy, Matplotlib, Seaborn documentation — [python.org](https://python.org), [pandas.pydata.org](https://pandas.pydata.org), [numpy.org](https://numpy.org), [matplotlib.org](https://matplotlib.org), [seaborn.pydata.org](https://seaborn.pydata.org)
- SciPy, Scikit-learn, Statsmodels, PuLP documentation — [docs.scipy.org](https://docs.scipy.org), [scikit-learn.org](https://scikit-learn.org), [statsmodels.org](https://statsmodels.org), [coin-or.github.io/pulp](https://coin-or.github.io/pulp)
- Online Retail transaction dataset — OnlineRetail.csv (dataset provided for this project)